



Práctica Banco Nexus (Etapa 1)

≡ Materia	Sistemas Distribuidos
≡ Etiquetas	actividad uabcs
🕒 Creado	May 12, 2026 3:05 PM



Ariel Felixmarco Perez Iglesias
Omar Armando Urquidez Castro
Jose Ernesto Guluarte Frias
Jose Carlos Victorio Coronado

Documentación de MongoDB e Infraestructura Local

| Responsable: Omar Urquidez

1. Propósito del documento

Este documento describe cómo preparar MongoDB localmente para el proyecto Banco Nexus, cómo validar la conexión simple de la etapa actual y cómo verificar que todos los integrantes trabajen con los mismos datos base.

En el estado actual del proyecto no se ha configurado réplica ni replica set. La ejecución se realiza sobre una instancia local de MongoDB por integrante.

2. Alcance del rol

Responsabilidades cubiertas en esta etapa:

- Preparar la instalación local de MongoDB;
- Documentar la instalación para los 4 nodos del equipo;
- Dejar listo un entorno de conexión simple;
- verificar que la base de datos cargue correctamente;
- Documentar problemas frecuentes y su solución.

3. Entorno de conexión actual

La aplicación usa una conexión local simple a MongoDB.

Cadena base de conexión:

```
mongodb://localhost:27017
```

Nombre de la base de datos: `"banco_nexus"`

El backend toma estos valores desde `backend/db.js` y permite sobrescribirlos con variables de entorno:

- **MONGO_URI**
- **DB_NAME**

4. Instalación local de MongoDB

El método recomendado para el equipo es ejecutar MongoDB con Docker, porque es el entorno usado por el proyecto y evita diferencias de instalación entre Windows, macOS y Linux.

También se documentan instalaciones nativas como alternativa, pero las verificaciones principales de este documento asumen el uso del contenedor Docker `banco_nexus_db`.

4.1 Instalación con Docker

Requisitos:

- Tener Docker Desktop o Docker Engine instalado;
- Tener Docker en ejecución antes de levantar la base de datos;
- Ejecutar los comandos desde la raíz del proyecto.

Crear y levantar el contenedor de MongoDB:

```
docker run -d --name banco_nexus_db -p 27017:27017 mongo:latest
```

Esto crea un contenedor llamado `banco_nexus_db` y expone MongoDB en el puerto local **27017**, por lo que la aplicación puede seguir usando `"mongodb://localhost:27017"`

Verificación del contenedor:

```
docker ps --filter name=banco_nexus_db
docker exec banco_nexus_db mongosh --eval "db.runCommand({ ping: 1 })"
```

Si el comando de `ping` responde correctamente, MongoDB está activo dentro del contenedor y disponible para el backend en `"localhost:27017"`.

4.2 Windows

Alternativa nativa si no se usa Docker:

1. Descargar MongoDB Community Server desde el sitio oficial de MongoDB.
2. Elegir la versión estable disponible para Windows.
3. Ejecutar el instalador.
4. Activar la opción de instalar MongoDB como servicio si está disponible.
5. Verificar que ``mongod`` y ``mongosh`` queden accesibles desde terminal.

Comandos de verificación:

```
mongod --version
```

```
mongosh --version
```

4.3 macOS

Alternativa nativa si no se usa Docker.

Usando Homebrew:

```
brew tap mongodb/brew  
brew install mongodb-community  
brew services start mongodb-community
```

Verificación:

```
mongod --version  
mongosh --version
```

4.4 Linux

Alternativa nativa si no se usa Docker.

La instalación depende de la distribución, pero en general se recomienda:

1. Agregar el repositorio oficial de MongoDB
2. Instalar MongoDB Community
3. Iniciar el servicio `mongod`
4. Validar acceso con `mongosh`.

Verificación:

```
mongod --version  
mongosh --version
```

5. Preparación del proyecto

Como apoyo operativo del proyecto, también se definió un archivo `Makefile` en la raíz del repositorio. Su propósito es dejar alias simples para comandos frecuentes, de modo que los integrantes puedan ejecutar tareas útiles desde la raíz sin tener que recordar o escribir comandos largos.

Esto ayuda especialmente a estandarizar el trabajo con Docker y con la carga inicial de datos.

Ejemplos disponibles en el `Makefile` :

- `make db-install`
- `make db-start`
- `make db-stop`
- `make db-seed`
- `make backend-install`
- `make backend-start`

Con MongoDB en ejecución, preparar el backend:

```
cd backend  
npm install
```

Después cargar los datos iniciales:

```
make db-seed # (node backend/seed.js)
```

6. ¿Qué hace el script de inicialización?

El archivo `backend/seed.js` realiza las siguientes acciones:

1. se conecta a MongoDB;
2. limpia las colecciones existentes;
3. inserta clientes, cuentas y transacciones;
4. crea índices útiles para consulta;
5. imprime un resumen de carga.

La carga actual contempla:

- 15 clientes;
- 15 cuentas;
- un conjunto de transacciones de prueba.

7. Estructura de la base de datos

Colecciones creadas:

- Clientes
- Cuentas
- Ttransacciones

Relación lógica:

```
clientes.curp -> cuentas.cliente -> transacciones.cuenta
```

Índices creados por el script:

- `clientes.curp` único
- `cuentas.cuenta` único
- `cuentas.cliente`
- `transacciones.cuenta`
- `transacciones.fecha`

8. Verificación de carga de datos

Como el método principal es Docker, las consultas se hacen entrando a `mongosh` dentro del contenedor, aunque si se tiene instalado de manera global (nativa), funcionaría igual.

```
docker exec -it banco_nexus_db mongosh
```

Seleccionar la base:

```
use banco_nexus
```

Validar conteos:

```
db.clientes.countDocuments()  
db.cuentas.countDocuments()  
db.transacciones.countDocuments()
```

Validar una cuenta de ejemplo:

```
db.cuentas.findOne({ cuenta: "1000000001" })
```

Validar transacciones de una cuenta:

```
db.transacciones.find({ cuenta: "1000000001" }).sort({ fecha: -1 })
```

También se puede ejecutar una verificación rápida sin entrar a la consola interactiva:

```
docker exec banco_nexus_db mongosh banco_nexus --eval "db.clientes.countDocuments()  
( )"  
docker exec banco_nexus_db mongosh banco_nexus --eval "db.cuentas.countDocuments()"  
docker exec banco_nexus_db mongosh banco_nexus --eval "db.transacciones.countDocuments()"
```

9. Verificación desde el backend

Iniciar el servidor:

```
cd backend  
npm start
```

Pruebas recomendadas:

```
curl http://localhost:3001/health  
curl http://localhost:3001/api/clientes  
curl http://localhost:3001/api/cuenta/1000000001  
curl http://localhost:3001/api/historial/1000000001
```

Estas pruebas permiten confirmar:

- que MongoDB está activo;

- que el backend se conectó correctamente;
- que las colecciones pueden consultarse;
- que la cuenta y sus movimientos están disponibles.

10. Verificación entre los 4 nodos

Como en esta etapa no existe réplica automática, la consistencia entre integrantes se valida levantando el mismo contenedor Docker, ejecutando el mismo script de carga en cada nodo y comparando resultados.

Pruebas sugeridas para cada integrante:

```
make db-start
make db-seed
docker exec -it banco_nexus_db mongosh banco_nexus
```

10.1 conteos

```
db.clientes.countDocuments()
db.cuentas.countDocuments()
db.transacciones.countDocuments()
```

10.2 Consulta ordenada de clientes

```
db.clientes.find(\{\}, \{ _id: 0, nombre: 1, curp: 1 \}).sort(\{ nombre: 1 \})
```

10.3 Consulta ordenada de cuentas

```
db.cuentas.find(\{\}, \{ _id: 0, cuenta: 1, cliente: 1, saldo: 1 \}).sort(\{ cuenta: 1 \})
```

10.4 Consulta de una cuenta de referencia

```
db.transacciones.find(\{ cuenta: "1000000001" \}).sort(\{ fecha: 1 \})
```

Si todos ejecutan el mismo `seed.js`, los resultados base deben ser equivalentes.

11. Problemas comunes y soluciones

Problema 1: Docker no está disponible

Causa probable:

- Docker Desktop o Docker Engine no está instalado;
- Docker no está iniciado
- el comando "docker" no está en el PATH.

Solución:

- instalar o abrir Docker
- reiniciar la terminal
- confirmar con `docker --version`
- volver a ejecutar ``make db-install`` o `make db-start`.

Problema 2: no conecta en "localhost:27017"

Causa probable:

- el contenedor `banco_nexus_db` no está corriendo;
- el puerto "27017" ya está ocupado por otra instancia de MongoDB.

Solución:

- revisar `docker ps --filter name=banco_nexus_db`;
- iniciar con `make db-start`;
- si el contenedor no existe, crearlo con ``make db-install``;
- volver a probar con `docker exec banco_nexus_db mongosh --eval "db.runCommand({ ping: 1 })"`.

Problema 3: el backend responde que la base no está disponible

Causa probable:

- el contenedor de MongoDB está apagado;
- el backend inició antes de que la conexión estuviera lista;
- la URI es incorrecta.

Solución:

- confirmar que el contenedor está activo con `docker ps --filter name=banco_nexus_db`;
- revisar que **MONGO_URI** apunte a "mongodb://localhost:27017";
- reiniciar el backend.

Problema 4: el frontend no muestra datos

Causa probable:

- el backend no está corriendo;
- la cuenta consultada no existe;
- la carga de datos inicial no se ejecutó.

Solución:

- revisar `curl http://localhost:3001/health`;
- volver a ejecutar `make db-seed`;
- probar con una cuenta conocida como "1000000001".

Problema 5: datos distintos entre integrantes

Causa probable:

- un integrante realizó depósitos o retiros después de cargar la base;
- alguien trabaja con una base anterior o distinta.

Solución:

- iniciar el contenedor con `make db-start`;
- volver a ejecutar `make db-seed` para reiniciar el estado base;
- comparar de nuevo conteos y cuentas de referencia.

12. Bitácora

Actividad	Estado	Evidencia
Documentación de instalación local de MongoDB con Docker	Completado	Este archivo
Definición del entorno simple de conexión	Completado	<code>backend/db.js</code>
Validación de base de datos local	Completado	consultas en <code>mongosh</code> dentro de <code>banco_nexus_db</code>
Verificación desde el backend	Completado	rutas <code>/health</code> , <code>/api/clientes</code> , <code>/api/cuenta/:cuenta</code>
Registro de problemas frecuentes	Completado	sección 11

13. Conclusión

La infraestructura de la etapa actual cumple con el objetivo de dejar a los cuatro integrantes trabajando sobre una instancia local de MongoDB ejecutada con Docker, con una base de datos común y verificable. La conexión está preparada de forma simple y reutilizable, el script de inicialización permite reconstruir el estado base del sistema y la aplicación puede validar el acceso a las colecciones desde el backend y desde la interfaz.

La configuración distribuida con réplica puede plantearse como evolución para una etapa posterior, pero no forma parte del alcance técnico implementado en el repositorio actual.